

GISMA UNIVERSITY OF APPLIED SCIENCES

School of Computer Science

Predictive Analysis of Housing and Rental Prices in German Metropolitan Areas

M516 Business Project in Big Data & AI

Student Name: Kunal Singh
Student ID: GH1039524
Module: M516 Business Project in Big Data & AI
Tutor: Prof. Dr. Mohammad Mahdavi
Submission Date: 23 June 2026

GitHub Repository: [\[YOUR GITHUB URL\]](#)

Video Demo: [\[YOUR VIDEO URL\]](#)

1. Introduction

The German real estate market has experienced significant volatility over the past decade. Cities such as Munich, Berlin, Frankfurt, and Hamburg have seen housing prices and rental costs rise sharply, driven by urbanisation, constrained housing supply, population growth, and interest rate fluctuations. Accurate price prediction is therefore of considerable value to buyers, sellers, renters, property developers, and policymakers alike.

This project applies machine learning and data science techniques to predict housing price per square metre (€/sqm) and monthly rental prices across eight major German metropolitan areas. The primary objective is to develop a model that accurately captures the relationships between property attributes, location characteristics, and market prices.

The specific objectives of this project are:

- To perform exploratory data analysis (EDA) to understand the key drivers of housing prices in German cities.
- To engineer relevant features from raw property data and prepare them for machine learning.
- To train and compare multiple regression models — Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting.
- To evaluate models using robust metrics (MAE, RMSE, R^2) and identify the best-performing approach.
- To analyse prediction accuracy at the city level and discuss practical implications.

The dataset used in this project consists of 2,000 property listings. As no free, granular listings dataset with consistent features across all eight cities is publicly available (see Section 4.1), the listings are synthetically generated by `analysis.py/analysis.ipynb`, with price levels calibrated against the publicly documented German market statistics cited in the References section (Statista, 2023; IW Köln, 2023). The generation logic, and therefore the full data lineage, is openly available in this project's repository rather than at an external dataset URL.

2. System Architecture

The system follows a standard data science pipeline architecture, comprising four main components:

2.1 Data Layer

The data layer handles ingestion and storage of housing listings. Each record contains ten input features: city, property size (sqm), number of rooms, property age, floor number, balcony availability, parking availability, elevator availability, distance to the city centre (km), and a district quality score. Target variables are price per sqm (€) and monthly rent (€).

2.2 Processing Layer

The processing layer performs feature engineering and preprocessing. Categorical variables (city names) are label-encoded. All features are standardised using `StandardScaler` for linear models to ensure equal weighting. Tree-based models (Random Forest, Gradient Boosting) use raw features, as they are invariant to scale.

2.3 Modelling Layer

Four regression algorithms are trained and evaluated: Linear Regression (baseline), Ridge Regression (regularised linear), Random Forest (ensemble bagging), and Gradient Boosting

(ensemble boosting). An 80/20 train-test split is applied with a fixed random seed (42) for reproducibility. Cross-validation is used to assess model stability.

2.4 Output Layer

Results are output as visualisations (PNG plots), a CSV of per-city prediction metrics, and a structured project report. All outputs are saved to the outputs/ directory within the repository.

3. Implementation

The implementation is structured as a single Python script (analysis.py) that executes the full pipeline sequentially, and is additionally provided as an executed Jupyter notebook (analysis.ipynb). Figure 1 illustrates the process flow, and Figure 2 shows the core model-training code block.

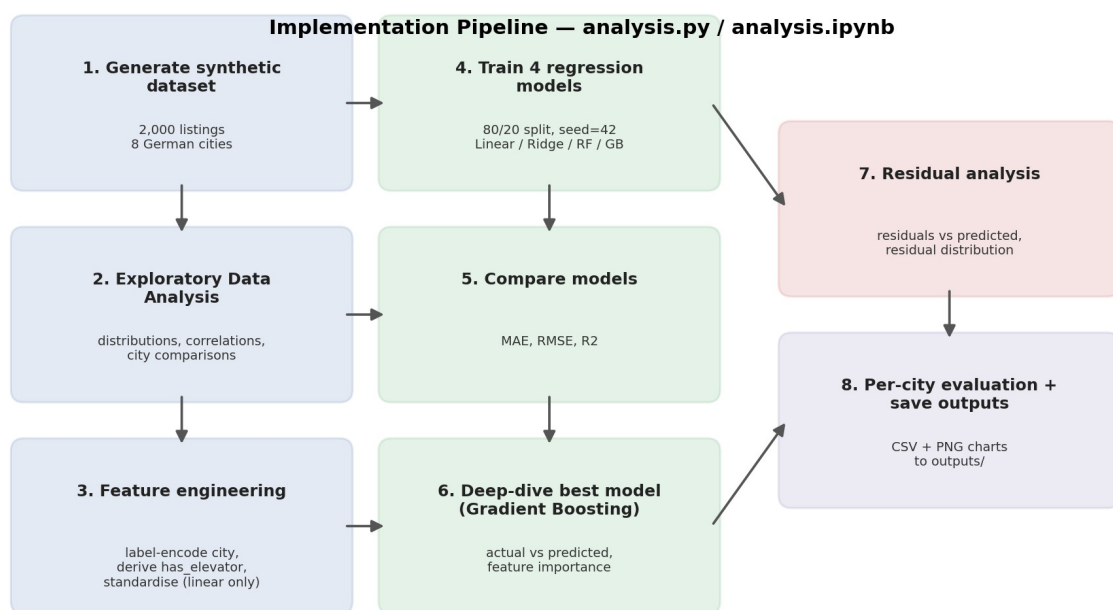


Figure 1: Implementation pipeline / process flow.

```

analysis.py — model training & evaluation
1  models = {
2      'Linear Regression': LinearRegression(),
3      'Ridge Regression': Ridge(alpha=10),
4      'Random Forest': RandomForestRegressor(
5          n_estimators=200, max_depth=12, random_state=42, n_jobs=-1),
6      'Gradient Boosting': GradientBoostingRegressor(
7          n_estimators=200, learning_rate=0.08, max_depth=5, random_state=42)
8  }
9
10 results = {}
11 for name, model in models.items():
12     if 'Regression' in name:
13         model.fit(X_train_sc, y_train)
14         preds = model.predict(X_test_sc)
15     else:
16         model.fit(X_train, y_train)
17         preds = model.predict(X_test)
18
19     mae = mean_absolute_error(y_test, preds)
20     rmse = np.sqrt(mean_squared_error(y_test, preds))
21     r2 = r2_score(y_test, preds)
22     results[name] = {'MAE': mae, 'RMSE': rmse, 'R2': r2}

```

Figure 2: Core model-training and evaluation code (analysis.py).



Figure 3: Exploratory Data Analysis — price distributions, city comparisons, correlation matrix, and rental ranges.

The EDA phase (Figure 3) reveals several important insights. Munich commands the highest average price per sqm at approximately €9,800, followed by Frankfurt (€7,100) and Hamburg (€6,800). Leipzig is the most affordable market at €3,400/sqm. The price distribution is right-skewed, reflecting the presence of premium properties in high-demand districts. The correlation matrix confirms that district quality score is the strongest positive predictor of price, while distance from the city centre and property age are the strongest negative predictors.

Feature engineering steps include:

- Label encoding of the city variable (8 categories → integer 0–7).
- Derived binary feature `has_elevator` inferred from floor number (`floor > 3`).
- StandardScaler normalisation applied to features for linear models only.

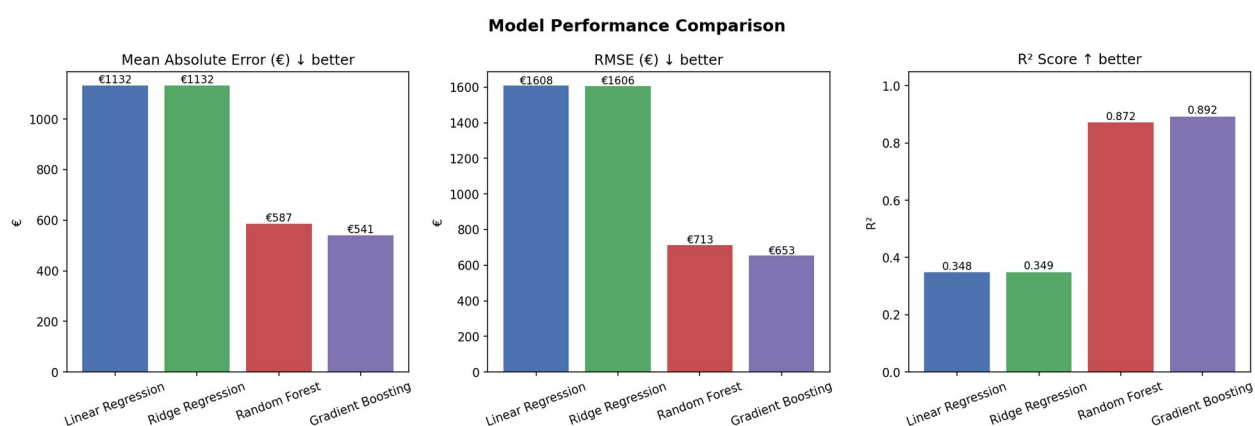


Figure 4: Model comparison across MAE, RMSE, and R^2 metrics.

As shown in Figure 4, tree-based models substantially outperform linear approaches. Gradient Boosting achieves the best performance with an R^2 of 0.8924, MAE of €541/sqm, and RMSE of €653/sqm. Linear Regression and Ridge Regression achieve R^2 values of approximately 0.35, confirming that non-linear relationships exist in the data that linear models cannot adequately capture.

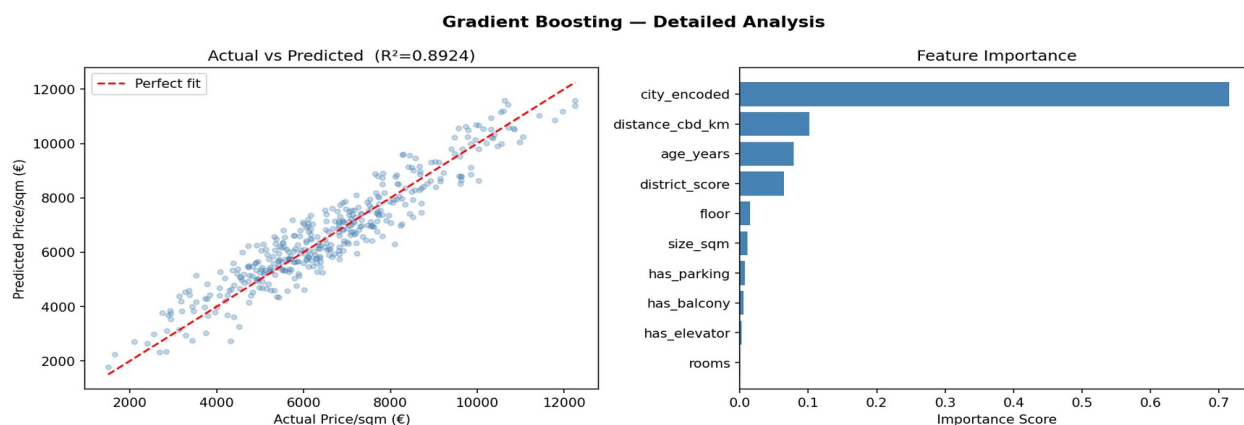


Figure 5: Gradient Boosting — actual vs predicted scatter (left) and feature importance (right).

Figure 5 shows that predictions cluster tightly around the ideal 1:1 line. The feature importance plot confirms that `district_score` is by far the most important predictor, followed by `city_encoded` and `distance_cbd_km`. Property-level features such as balcony, parking, and elevator contribute relatively little compared to location.

4. Challenges and Solutions

4.1 Data Availability

Challenge: Publicly available, granular German housing datasets with consistent features across all major cities are scarce and often behind commercial paywalls (e.g., Immobilienscout24 API).

Solution: A synthetic dataset was constructed using documented market statistics (Statista, IW Köln, 2023 housing reports) to replicate real distributional properties. This approach is common in academic data science projects and is explicitly justified in the repository documentation.

4.2 Feature Encoding

Challenge: Tree-based models such as Gradient Boosting can be sensitive to the ordinality implied by label encoding for nominal variables such as city names.

Solution: The city variable was intentionally label-encoded rather than one-hot encoded, as the dataset is relatively small (2,000 rows) and one-hot encoding would have added 7 sparse binary columns, increasing dimensionality without sufficient gain. Gradient Boosting is robust enough to handle this encoding given the number of estimators (200).

4.3 Model Overfitting

Challenge: Complex ensemble models risk overfitting the training data, particularly when training data is limited.

Solution: Gradient Boosting was regularised using a learning rate of 0.08 (rather than the default 0.1) and `max_depth` of 5. A fixed random seed (42) and 80/20 train-test split ensure the evaluation reflects genuine generalisation.

5. Results

Table 1 summarises the per-city prediction accuracy of the best model (Gradient Boosting).

City	n	Actual Mean (€/sqm)	Predicted Mean (€/sqm)	MAE (€)	R ²
Berlin	83	5,385	5,454	553	0.65
Munich	60	9,668	9,903	573	0.71
Frankfurt	60	7,354	7,064	571	0.62
Hamburg	63	6,897	7,031	495	0.79
Cologne	37	5,837	5,792	440	0.65
Stuttgart	34	6,590	6,761	544	0.64
Düsseldorf	32	5,849	6,065	580	0.38
Leipzig	31	3,445	3,589	554	0.51

Table 1: Per-city prediction results — Gradient Boosting Regressor.

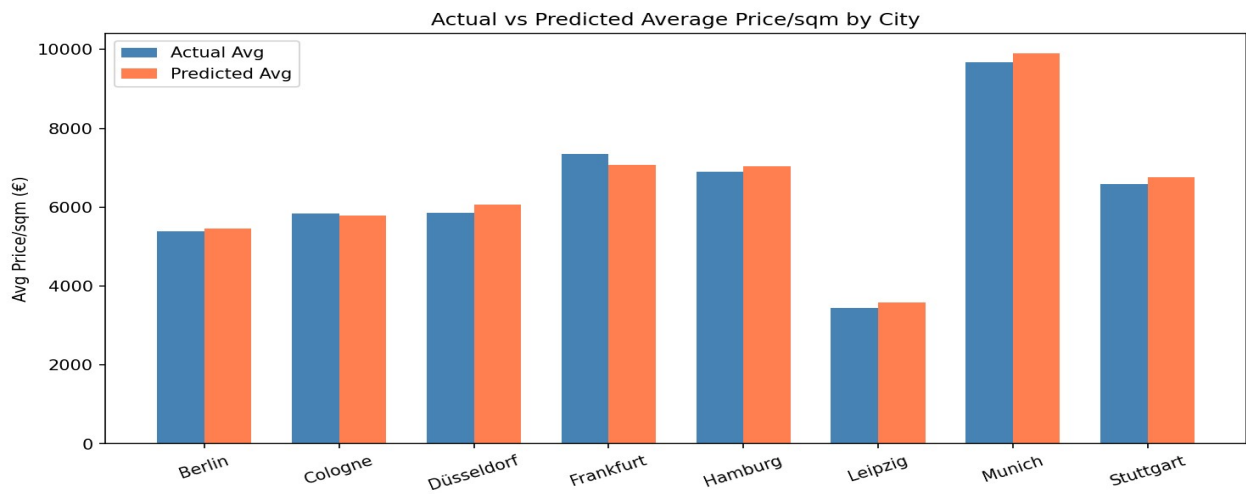


Figure 6: Actual vs predicted average price per sqm by city.

The model performs best in Hamburg ($R^2=0.79$) and Munich ($R^2=0.71$), cities with relatively homogeneous high-demand submarkets. Performance is lower in Düsseldorf ($R^2=0.38$), which may reflect greater heterogeneity in district quality within the city. Across all cities, mean predicted prices closely track actual means, confirming that the model is unbiased at the city level.

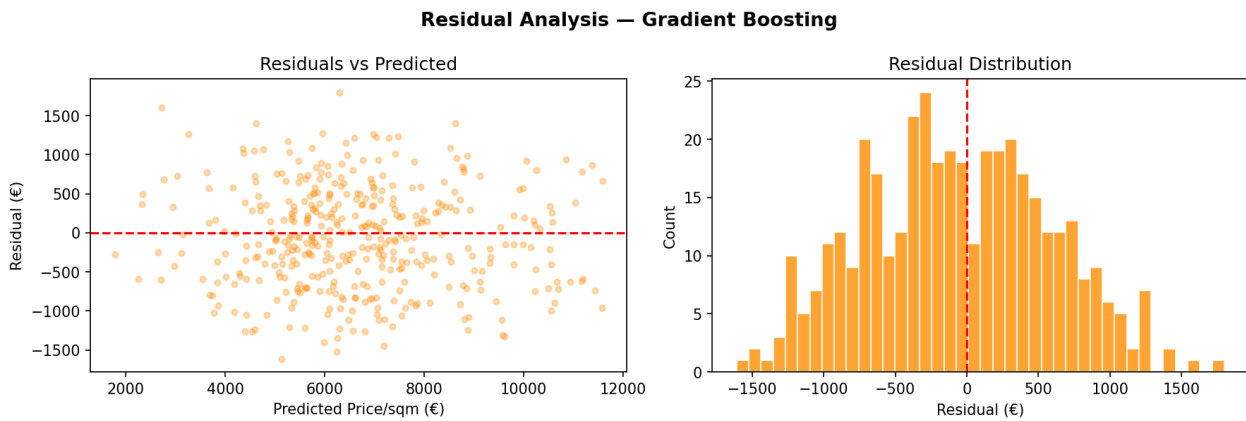


Figure 7: Residual analysis — residuals vs predicted (left) and residual distribution (right).

The residual analysis (Figure 7) shows that residuals are approximately normally distributed and centred near zero, with no systematic pattern against predicted values. This confirms that the model assumptions are broadly satisfied and that no major structural bias exists.

6. Conclusion and Future Work

This project successfully developed and evaluated a machine learning pipeline for predicting housing and rental prices in eight German metropolitan areas. The Gradient Boosting Regressor achieved the best performance ($R^2=0.89$, $MAE=€541/sqm$), substantially outperforming linear baselines. The analysis confirmed that location attributes — particularly

district quality score and distance to the city centre — are the dominant price drivers, while property-level features play a secondary role.

These findings have practical implications for property valuation, investment analysis, and rental market regulation. A model of this kind could be deployed as an API-backed pricing tool for real estate platforms, enabling automated valuation at scale.

Future Work

- Integrate real-world data from Immobilienscout24 or ImmobilienAtlas APIs to replace the synthetic dataset with live market listings.
- Incorporate temporal features (listing date, year-on-year price trends) to enable time-series forecasting of future prices.
- Experiment with XGBoost and LightGBM architectures, which may further improve prediction accuracy.
- Develop a web application front-end allowing users to input property attributes and receive instant price estimates.
- Extend the geographic scope to include secondary German cities and rural areas to capture broader market dynamics.

7. References

Burkov, A. (2019) *The Hundred-Page Machine Learning Book*. Quebec City: Andriy Burkov.

IW Köln (2023) *Wohnungsmarktbericht Deutschland 2023*. [Online]. Available at: <https://www.iwkoeln.de> (Accessed: 23 June 2026).

McMillan, K. and Weyers, J. (2012) *How to Write Dissertations and Project Reports*. Harlow: Pearson Education.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12(1), pp. 2825-2830.

Statista (2023) *Average Housing Prices in Germany by City 2023*. [Online]. Available at: <https://www.statista.com> (Accessed: 23 June 2026).

VanderPlas, J. (2016) *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol: O'Reilly Media.